

第二章 进程、线程与作业

提纲

2.1 多道程序设计

2.2 进程的引入

2.3 线程与轻进程

2.4 作业

提纲

2.1 多道程序设计

2.2 进程的引入

2.3 线程与轻进程

2.4 作业

多道程序设计目标

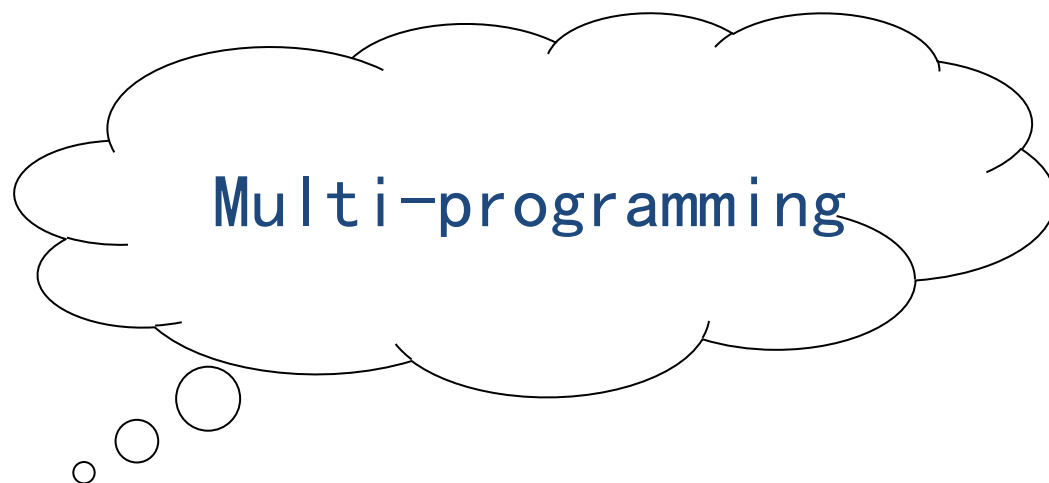
- 提高系统效率(吞吐量)

$$\text{吞吐量} = \frac{\text{作业道数}}{\text{全部处理时间}}$$

- 系统效率与系统资源的利用率有着密切的关系

2.1 多道程序设计

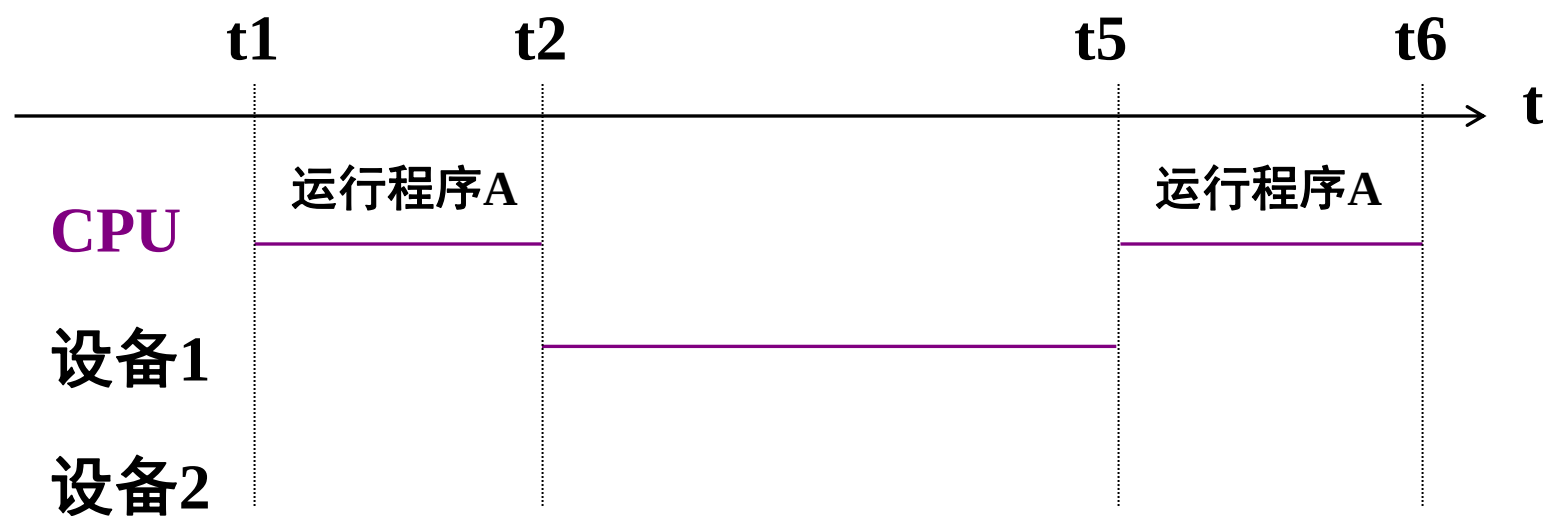
- 2.1.1 单道程序设计的缺点
- 2.1.2 多道程序设计的提出
- 2.1.3 多道程序设计的问题



2.1.1 单道程序设计的缺点

- 单道程序设计
 - 一次只允许一个程序进入系统的程序设计方法
- 缺点：资源利用率低.

2.1.1 单道程序设计的缺点(Cont.)

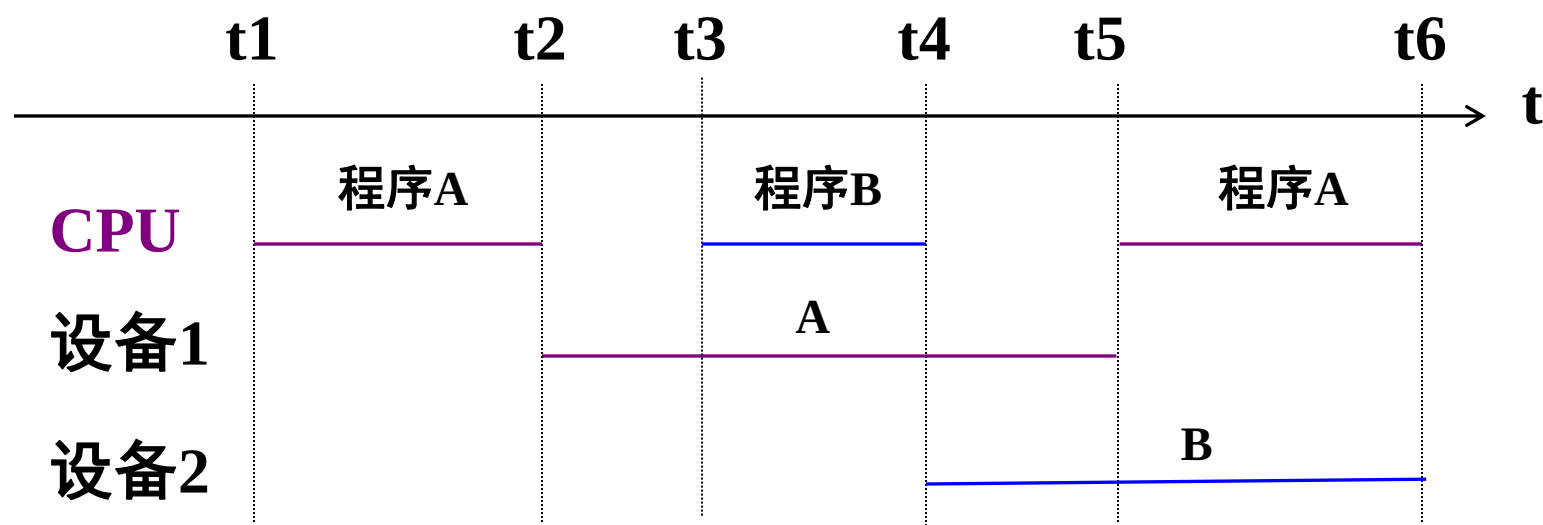


2.1.2 多道程序设计的提出

- 多道程序设计：多个程序同时进入系统并投入执行的一种程序设计方法
- 随着中断、通道、DMA控制器的引入，使得处理器与外围设备的并行成为可能.

2.1.2 多道程序设计的提出(Cont.)

- 提高处理器、设备、内存等各种资源的利用率，从而提高系统效率。



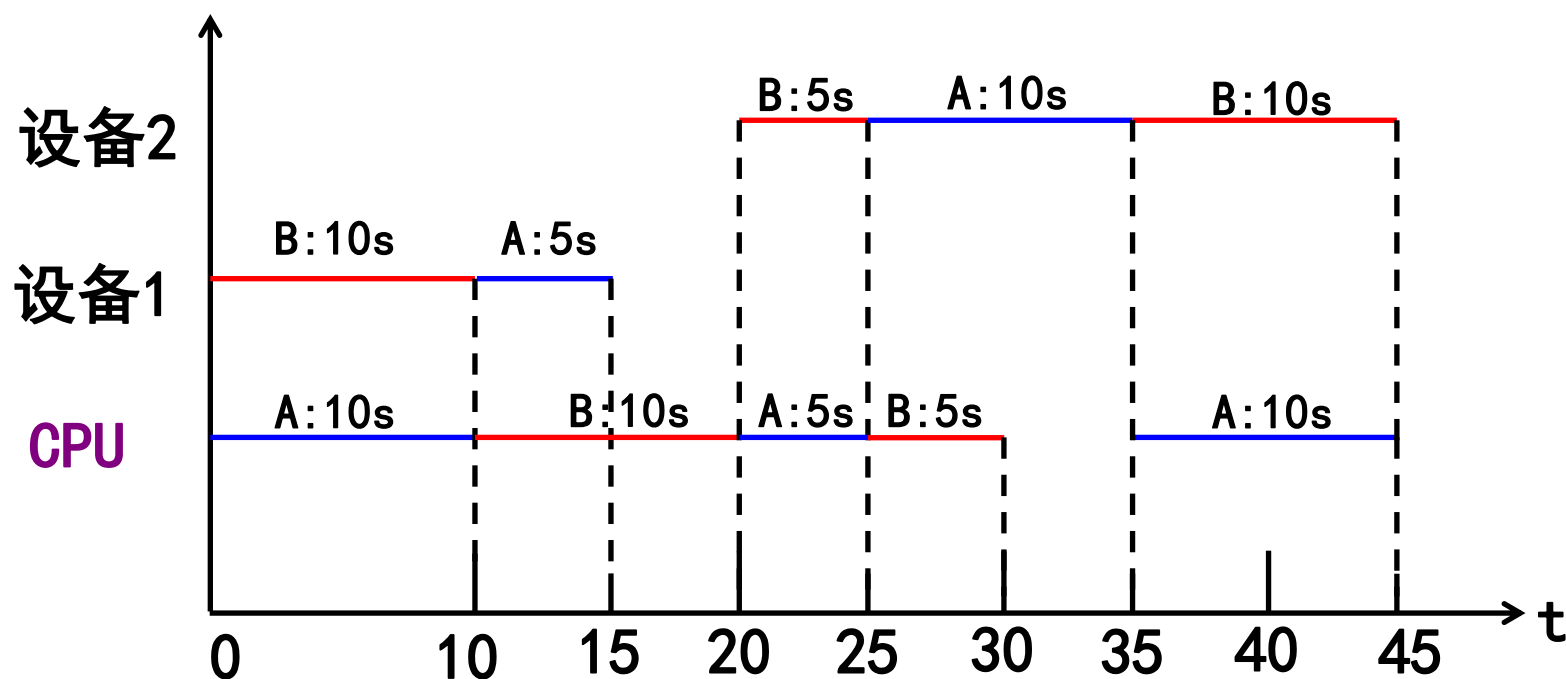
2.1.2 多道程序设计的提出(Cont.)

- 例，有两个程序A和B，A程序执行时所做的工作按次序需要的时间为：CPU计10s，设备1计5s，CPU计5s，设备2计10s，CPU计10s。B程序执行时间所做的工作按次序需要的时间为：设备1计10s，CPU计10s，设备2计5s，CPU计5s，设备2计10s。问在单道方式下和多道并发环境下执行A、B两个程序，CPU的利用率各为多少？
- 解：CPU的利用率=占有CPU总时间/运行总时间。在单道方式下，两个程序占用CPU的总时间为 $10+5+10+10+5=40s$ ，运行总时间 $10+5+5+10+10+10+10+5+5+10=80s$ ，所以单CPU利用率 $=40/80=50\%$ 。
- 在多道方式下，A和B2个程序占有CPU的总时间为 $10+10+5+5+10=40s$ ，运行总时间45s，所以CPU利用率 $=40/45=89\%$

2.1.2 多道程序设计的提出(Cont.)

A: CPU计10s, 设备1计5s, CPU计5s, 设备2计10s, CPU计10s

B: 设备1计10s, CPU计10s, 设备2计5s, CPU计5s, 设备2计10s



作业1

- 进程P对磁盘数据进行处理，其处理流程为：
 - ① 读入一块磁盘数据；
 - ② 处理该块数据；
 - ③ 把处理后的该块数据写到磁带上。
- 假设：进程P要处理3个磁盘块的数据，读一个磁盘块数据需要40ms，处理一块数据需要20ms，写一块磁带数据需要80ms，并假设系统只有进程P运行。在不考虑系统开销的情况下，
- 问 题：
 - (1) 进程P把3块数据处理流程都完成，画出CPU运行、磁盘输入、磁带输出的时序图；计算进程P等待设备I/O的时间、CPU的利用率。
 - (2) 为了提高资源利用率，进程P创建3个子进程分别处理3个数据块（创建进程的开销可忽略不计）。画出3个子进程并发运行的时序图，并画出3个子进程从创建到执行结束的状态转换图。要求标明状态转换原因。

2.1.2 多道程序设计的提出(Cont.)

思考：内存中的程序数量是否越多越好？

- 道数过少，系统资源利用率低
- 道数过多，系统开销 (system overhead) 增大，程序响应速度下降

2.1.3 多道程序设计的问题

- 处理器资源的管理
 - 程序个数>处理器个数(如何分配?)
- 存储资源的管理
 - 地址空间的相对独立性、共享性
 - 内存、外存(swap space)的分配与去配
- 设备资源管理
 - 分配策略：多个程序同时要求使用同一资源，如何分配
 - I/O控制.

提纲

2.1 多道程序设计

2.2 进程的引入

2.3 线程与轻进程

2.4 作业

2.2 进程的引入

- 2.2.1 进程的概念
- 2.2.2 进程状态及状态转换
- 2.2.3 进程控制块
- 2.2.4 进程的组成与上下文
- 2.2.5 进程的队列
- 2.2.6 进程的类型与特征
- 2.2.7 进程间相互联系与相互作用
- 2.2.8 进程的创建与撤销
- 2.2.9 进程与程序间的联系与差别

2.2 进程的引入(Cont.)

多道系统中的程序：

推进，暂停，推进，暂停，…….

暂停：保存现场 (PSW+PC, 寄存器)

推进：恢复现场 (寄存器, PSW+PC)

暂停原因：(1) 自身原因：等待资源，启动IO

(2) 剥夺CPU—给其它程序运行机会

2.2.1 进程的概念

- 定义：
 - 进程是程序的一次执行
 - 进程是可以参与并发执行的程序
 - 进程是程序和数据一道通过处理器执行时所发生的活动
 - 进程是具有一定独立功能的程序关于一个数据集合的一次运行活动.

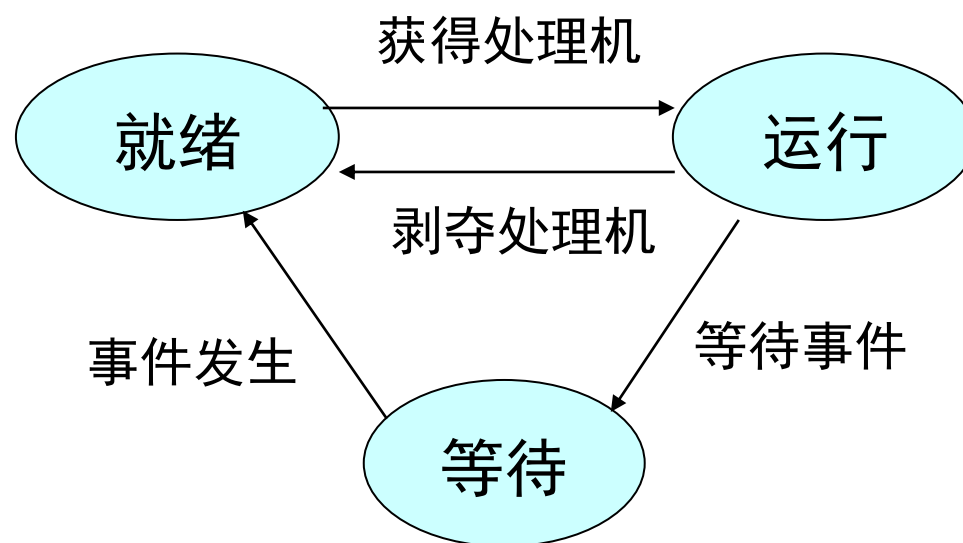
2.2.1 进程的概念(Cont.)

- 定义强调两个方面：
 - **动态**：执行中的程序。这是进程和程序的本质差异
 - **并发**：可与其它进程同时执行
 - **宏观同时**，“交替执行”，不要求多个CPU

2.2.2 进程状态及状态转换

- 2.2.2.1 进程状态(基本状态)
 - 运行态(RUN)：占有CPU正在向前推进
 - 就绪态(READY)：可以运行，但未得到CPU
 - 等待态(WAIT)：等待某一事件发生.

2.2.2.2 进程状态转换图



- 进程状态转换由**操作系统**完成，对用户是透明的
- 进程在其生存期内经过多次状态转换

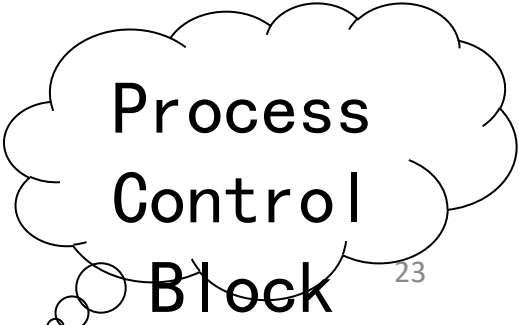
Keep in Mind

2.2.2.2 进程状态转换原因

- CPU调度
- 进程在运行过程中需要等待某一事件
- 进程等待的事件发生.

2.2.3 进程控制块(PCB)

- 标志进程存在的数据结构，其中保存系统管理进程所需的全部信息
- PCB内容：（不同系统不尽相同）
 - 进程标识(pid)
 - 进程状态
 - 现场信息
 - 调度参数
 - 所属用户(uid)
 - 家族联系
 - 地址信息
 - 打开文件
 - 消息队列指针
 - 进程队列指针



Process
Control
Block

2.2.3 进程控制块(Cont.)

■ PCB作用

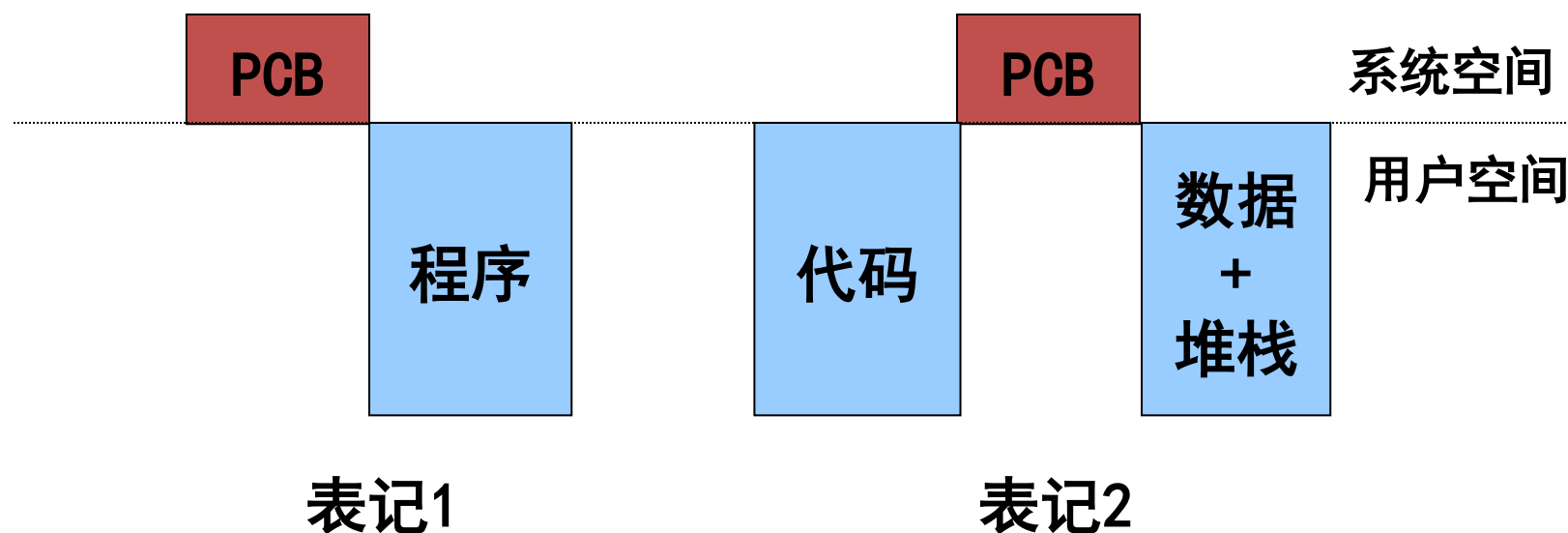
- 当调度某进程执行时，需要从该进程的PCB中查询其状态及优先级等参数
- 当调度到某进程后，根据PCB中的现场信息恢复现场，并根据PCB中的程序和数据内存地址找到程序和数据
- 进程执行过程中，当需要与其它进程通信时，也要访问PCB
- 当进程发生进程切换时，需要将现场信息从系统栈弹出，保存于PCB中
- 系统建立进程时建立PCB，撤销进程时撤销PCB.

2.2.4 进程的组成与上下文

- 进程的组成
 - 进程控制块 (process control block)
 - 建立进程 $\Rightarrow$ 建立PCB
 - 撤销PCB $\Rightarrow$ 撤销进程
 - 程序
 - 代码 (code)：实现相应的功能
 - 数据 (data)：静态数据+动态堆和动态栈
 - 堆栈 (stack+heap)
 - 栈：保存返回值、参数、断点、局部变量
 - 堆：动态变量

2.2.4 进程的组成与上下文(Cont.)

- 进程的表记



定义：进程的**程序**（代码和数据）称为**进程影像 (Process Image)**。

2.2.4 进程的组成与上下文(Cont.)

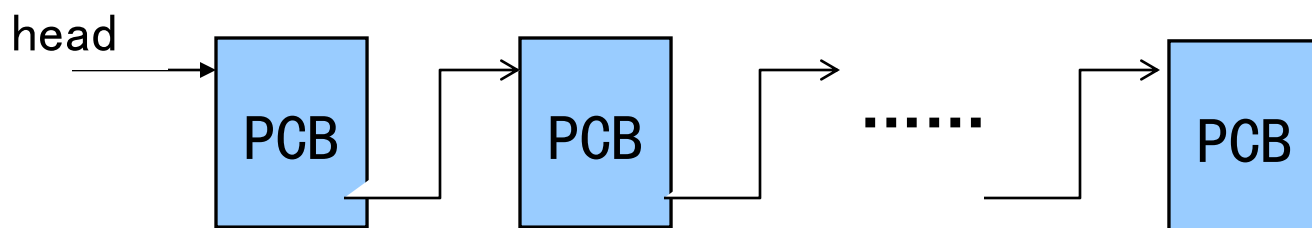
- 进程上下文 (Process Context)
 - PCB+程序
 - 系统环境：地址空间，系统栈，打开文件表，...
- 上下文切换 (context switch)
 - 由一个进程的上下文转到另外一个进程的上下文
- 系统开销 (system overhead)
 - 运行操作系统程序完成系统管理工作所花费的时间和空间

2.2.4 进程的组成与上下文(Cont.)

- 进程切换时需要保存的现场信息
 - 地址寄存器
 - 保存当前CPU所访问的内存单元的地址
 - 通用寄存器
 - 用于传送和暂存数据，也可参与算术逻辑运算，并保存运算结果
 - 浮点寄存器
 - 用于存储浮点数字，它决定着计算机的计算精度。
 - SP(系统栈指针)
 - PSW(程序状态字)
 - PC(指令计数器)
 - 打开文件表.

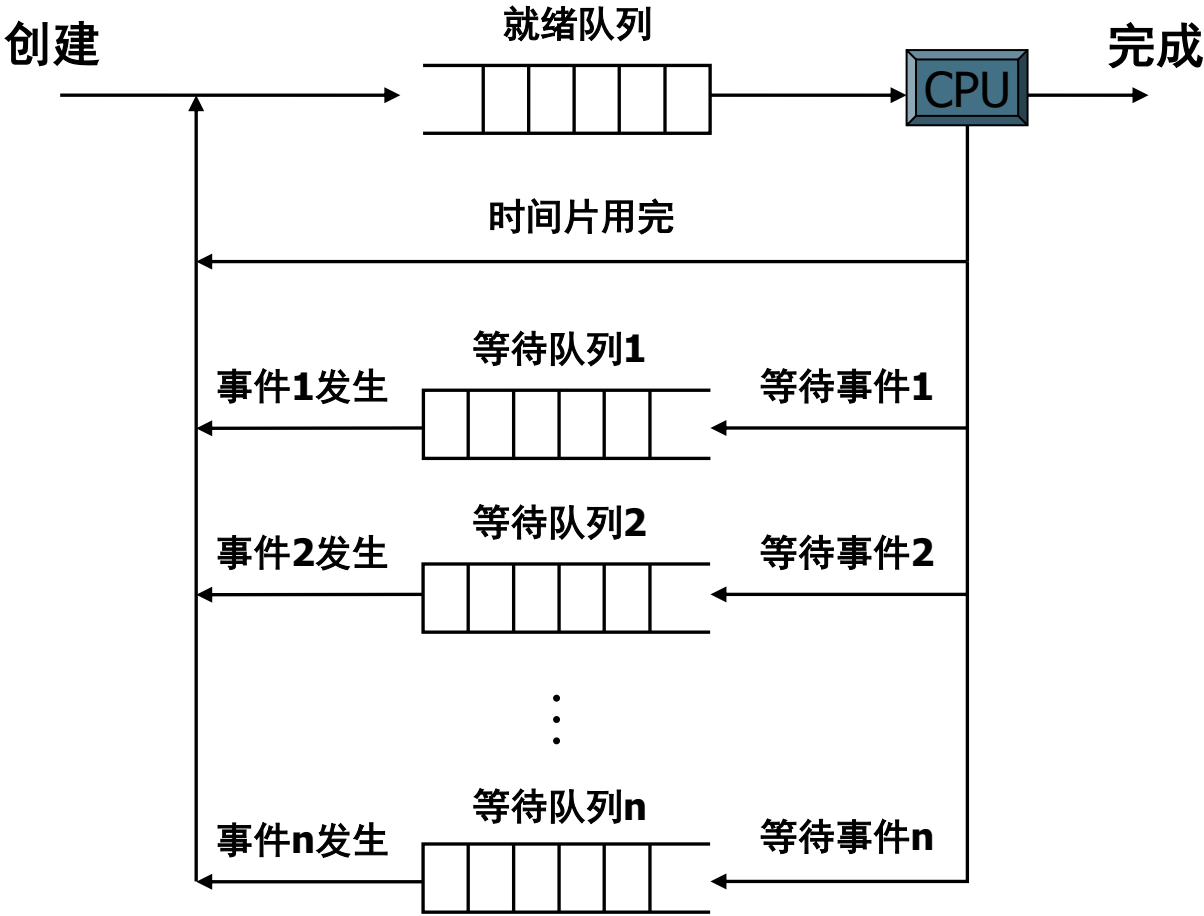
2.2.5 进程的队列

PCB构成的队列：（不一定FIFO，单向或双向）



1. 就绪队列：系统一个或若干个（根据调度算法确定）
2. 等待队列：每个等待事件一个
3. 运行队列：每个处理器一个

进程队列模型



2.2.6 进程的类型与特征

- 进程类型

- 系统进程

- 运行操作系统程序，管态，完成系统管理(服务)功能
 - 一个系统进程所完成的内容相对独立和具体，在进程生存周期保持不变，因而它们通常对应一个无限循环程序，在系统启动后便一直存在，直到系统关闭
 - 系统进程承担系统的管理和维护性任务，优先级高于一般用户进程
 - 例如：UNIX #0 -- sched, #1 -- init

- 用户进程

- 在操作系统之上运行的所有程序
 - 运行用户(应用)程序，为用户服务
 - 例如：UNIX vi, shell, cc.

2.2.6 进程的类型与特征(Cont.)

■ 进程的特征

- 并发性：可以与其它进程一道向前推进；
- 动态性：动态产生、消亡，生存期内动态变化；
- 独立性：一个进程是可以调度的基本单位；
- 交互性：同时运行的进程可能发生相互作用；
- 异步性：进程以各自独立，不可预知的速度向前推进；
- 结构性：每个进程有一个PCB。

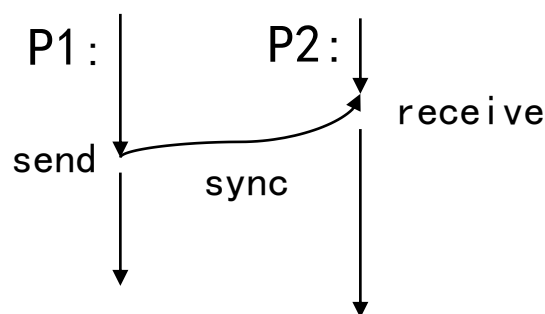
2.2.7 进程间相互联系与相互作用

- 相互联系
 - 相关进程
 - 同一家族的进程
 - 可以共享文件，需要相互通讯，协调推进速度...
 - 父进程可以监视子进程，子进程完成父进程交给的任务
 - 无关进程
 - 没有逻辑关系、同时执行的进程。
 - 有资源竞争关系，互斥、死锁、饿死。

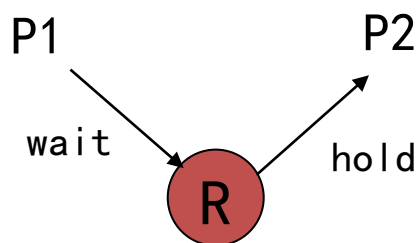
2.2.7 进程间相互联系与相互作用

■ 相互作用

1. 直接相互作用：发生在相关进程之间，进程之间不需要通过媒介而发生相互作用



2. 间接相互作用：发生在任何进程之间



2.2.8 进程的创建与撤销

- 进程创建
 - 向系统申请一个空闲PCB，并指定唯一的进程标识
 - 为新进程分配资源
 - 初始化新进程的PCB
 - 加载程序
 - 将PCB入就绪队列.

UNIX: `pid=fork()`, `exec(prog, args)`

2.2.8 进程的创建与撤销(Cont.)

- 引起进程创建的事件
 - 用户登录
 - 作业调度
 - 提供服务
 - 应用请求.

2.2.8 进程的创建与撤销(Cont.)

- 进程撤销
 - 从系统PCB表中找到被撤销进程的PCB
 - 检查被撤销进程的状态是否为执行状态。若是，则立即停止该进程的执行
 - 设置重新调度标志，以便在该进程撤销后将处理器分配给其它进程
 - 检查被撤销进程是否有子孙进程，若有子孙进程还应撤销该进程的子孙进程
 - 回收该进程占有的全部资源并回收其PCB.
- UNIX: `exit()` VS. `KILL`

2.2.8 进程的创建与撤销(Cont.)

- 引起进程撤销的事件
 - 正常结束
 - 异常结束
 - 外界干预.

2.2.8 进程的创建与撤销(Cont.)

- 进程汇聚

- Java语言中，父线程创建子线程后，可以执行join系统调用，等待子线程结束并与其汇聚，之后父线程继续执行。
- 用于数学计算，例如矩阵。

2.2.8 进程的创建与撤销(Cont.)

■ 进程等待

- 停止当前进程的执行，由于该进程正处于执行状态，故应停止该进程的执行
- 保存该进程的现场信息。为了使进程以后能够重新调度运行，应将进程的现场信息送入其PCB中保存
- 将进程状态改为等待.

2.2.8 进程的创建与撤销(Cont.)

■ 引起进程等待事件

■ 请求系统服务

- 当正在执行的进程请求操作系统服务时，而操作系统由于某种原因并不能立即满足该进程的要求时，该进程只能转为等待状态

■ 启动某种操作

- 当进程启动某种操作且该进程必须在该操作完成之后才能继续执行，必须先将进程等待

■ 新数据尚未到达

- 对于相互合作的进程，如果其中一个进程需要首先获取另一进程提供的数据才能继续，而其所需数据尚未到达

■ 无新工作可做

- 对于系统设置的具有特定功能的系统进程，在其完成任务后，会把状态设置为“等待”，以等待新任务的到达。

2.2.8 进程的创建与撤销(Cont.)

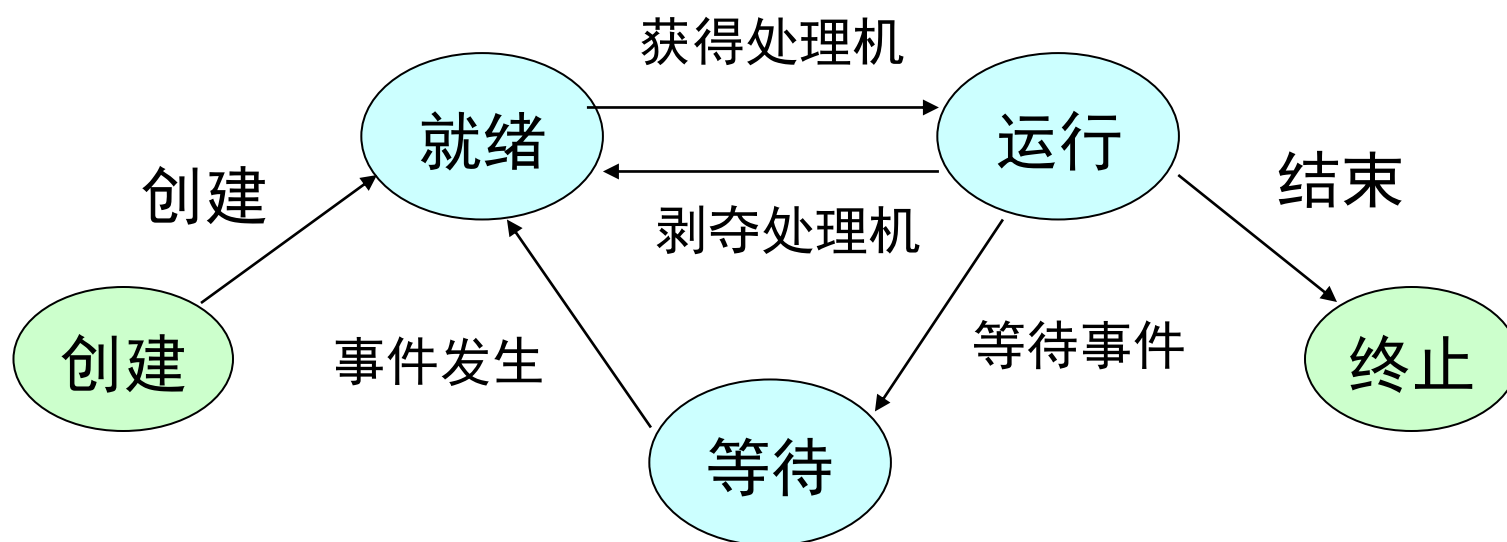
- 进程唤醒
 - 将被唤醒进程从相应的等待队列中移除
 - 将进程状态改为就绪，并将该进程插入就绪队列.

2.2.8 进程的创建与撤销(Cont.)

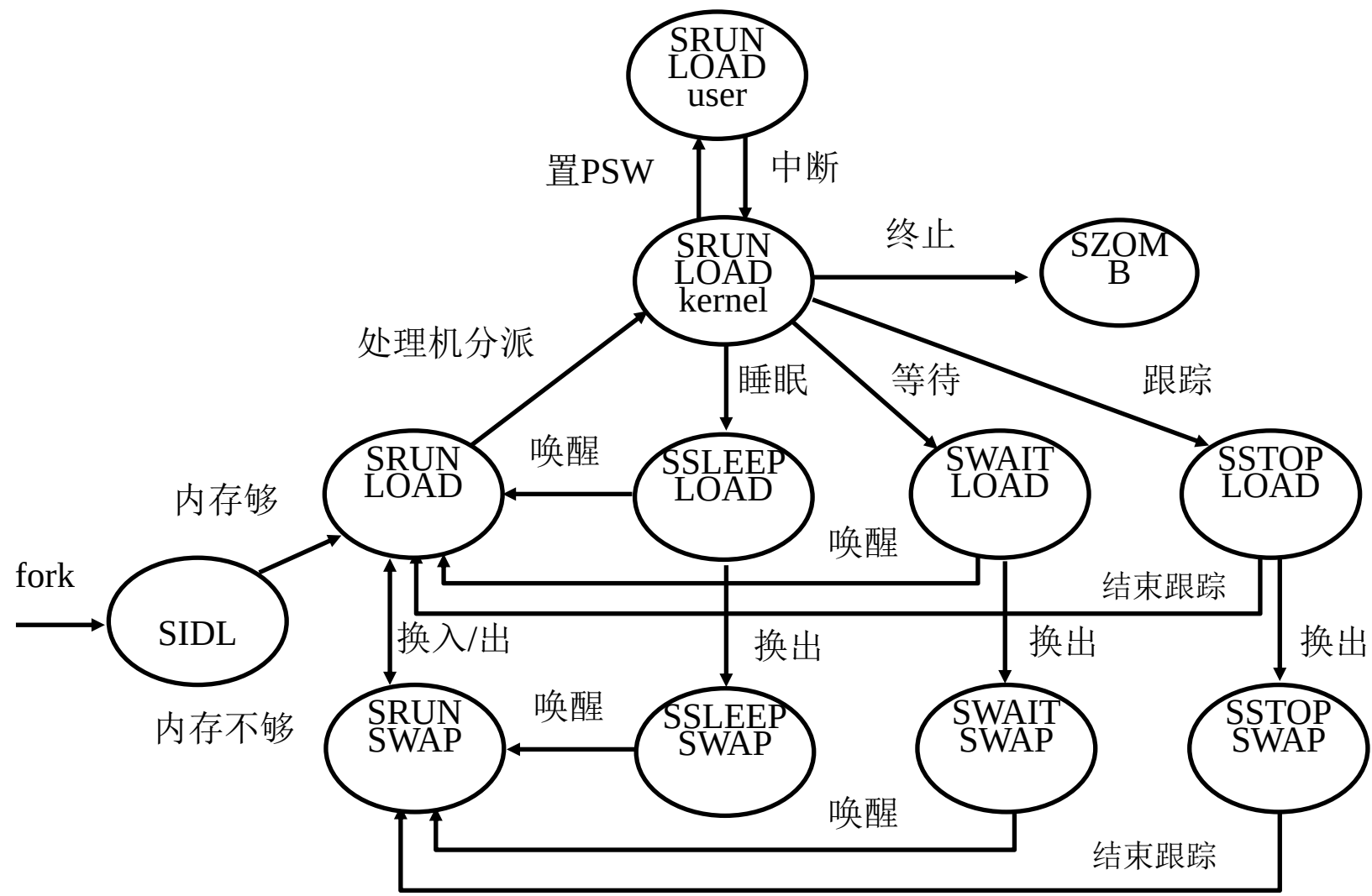
思考：父进程创建子进程与主程序调用子程序区别？

- 父进程创建子进程后，父进程与子进程可同时执行
- 主程序调用子程序后，主程序暂停在调用点，子程序开始执行，直到子程序执行完毕返回，主程序才开始执行

考虑生灭的进程状态转换图



UNIX进程状态及转换



UNIX 系统

- 内核

- 操作系统，向上层软件提供的界面是系统调用.

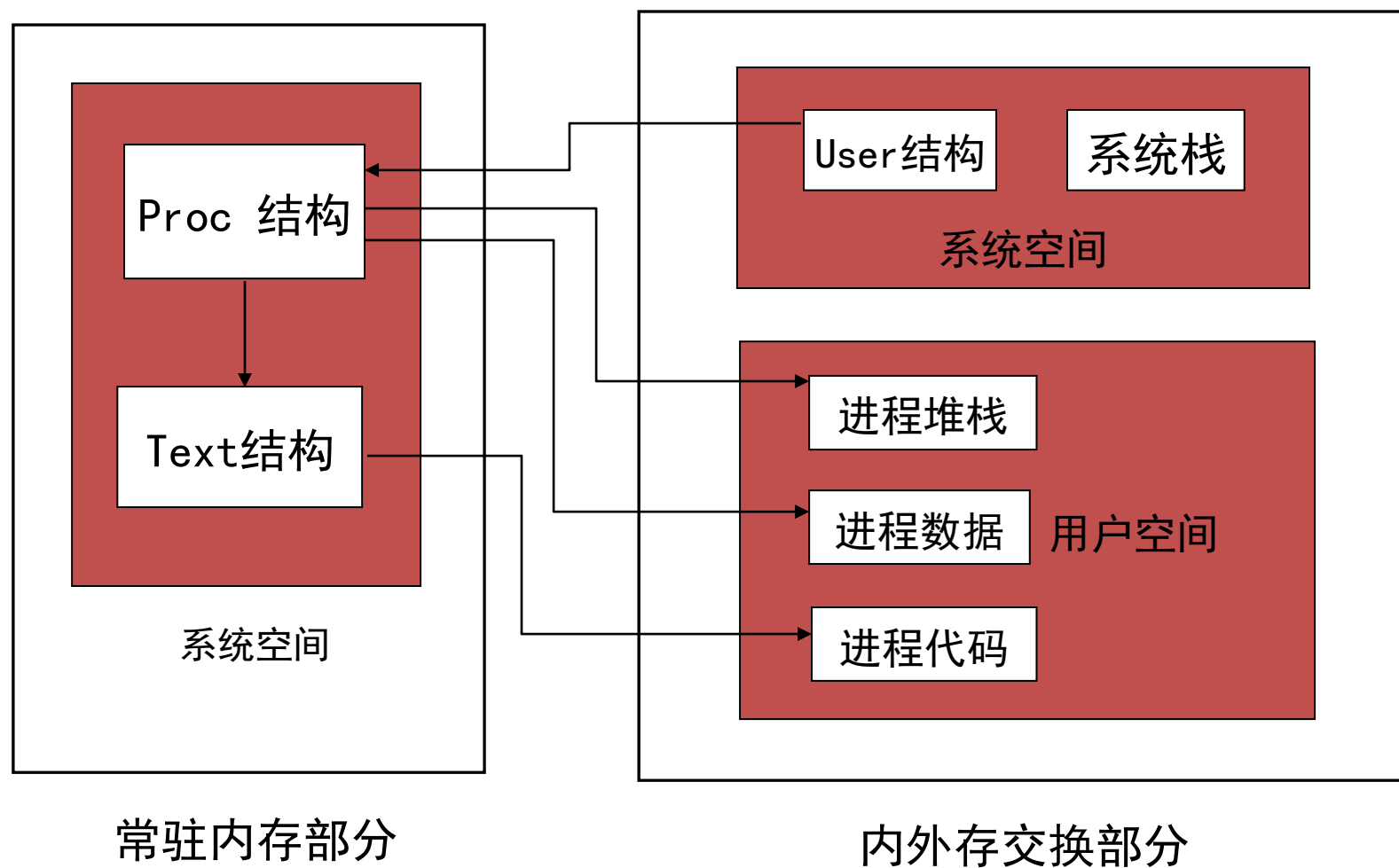
- 外壳

- 命令解释程序，为终端用户提供的界面是交互终端命令.

UNIX 进程

- *进程控制块*
 - Proc结构，常驻内存
 - User结构，可交换
- *程序*
 - 多个进程可共享的代码
 - 进程私用的数据和堆栈

UNIX 进程




```
struct proc
{
    char p_stat;    {SSLEEP,SWAIT,SRUN,SIDL, SSTOP,SZOMB}
    char p_flag;    {SLOAD,SSYS,SLOCK,SSWAP,SWTED}
    char p_pri;
    char p_sig;      //signal received
    char p_uid;      //user id
    char p_time;     //resident time for scheduling
    char p_cpu;      //cpu usage
    char p_nice;     //nice for scheduling
    char p_ttyp;     //controlling tty
    int  p_pid;      //unique process id
    int  p_ppid;     //parent process id
    int  p_addr;     //address of swappable image
    int  p_size;     //size of swappable image (*64bytes)
    int  p_wchan     //event process is waiting
    int  *p_textp;   //pointer to text structure
} proc[NPROC];
```

```
struct user
{
    int    u_rsav[2]
    int    u_fsav[25]
    .....
    char    u_uid;           // effective user id
    char    u_gid;           // effective group id
    char    u_ruid;          // real user id
    char    u_rgid;          // real group id
    int     u_procp;          // pointer to proc structure
    int     *u_base;          // base address for IO
    int     *u_count;         // bytes remaining for IO
    int     *u_offset[2];     // offset in file for IO
    int     *u_cdir;          // pointer to inode of current dir
    int     *u_pdir;          // inode of parent directory
    int     u_uisa[16];       // prototype segmentation address
}
```

```
int    u_uisd[16];    // prototype segmentation descript
int    u_ofile[NOFILE];    // pointers to file struct
int    u_arg[5];      // arguments to system call
int    u_tsize;       // text size (*64)
int    u_dsize;       // data size (*64)
int    u_ssize;       // stack size (*64)
int    u_utime;       // this process user time
int    u_stime;       // this process system time
int    u_cutime[2];   // child process user time
int    u_cstime[2];   // child process system time
.....
}
```

```
struct text
{
    int    x_daddr;        //disk address of segment
    int    x_caddr;        //core address if loaded
    int    x_size;         //*64
    int    *x_iptr;        //inode of prototype
    char    x_count;        //reference count
    char    x_ccount;       //number of loaded references
}
```

UNIX有关进程的系统调用命令

- 相关系统调用

- 创建子进程

- `Pid = fork()`

- 子进程是父进程的复制品

- 返回值：父进程为子进程编号，子进程为0，出错为-1

- 子进程与父进程并发运行

- 加载并执行新程序

- `execl(prog, arg0, ..., argn-1, 0)`

- 以`arg0, ..., argn-1`为参数执行`prog`程序

- 覆盖原来程序，从第一条指令开始执行

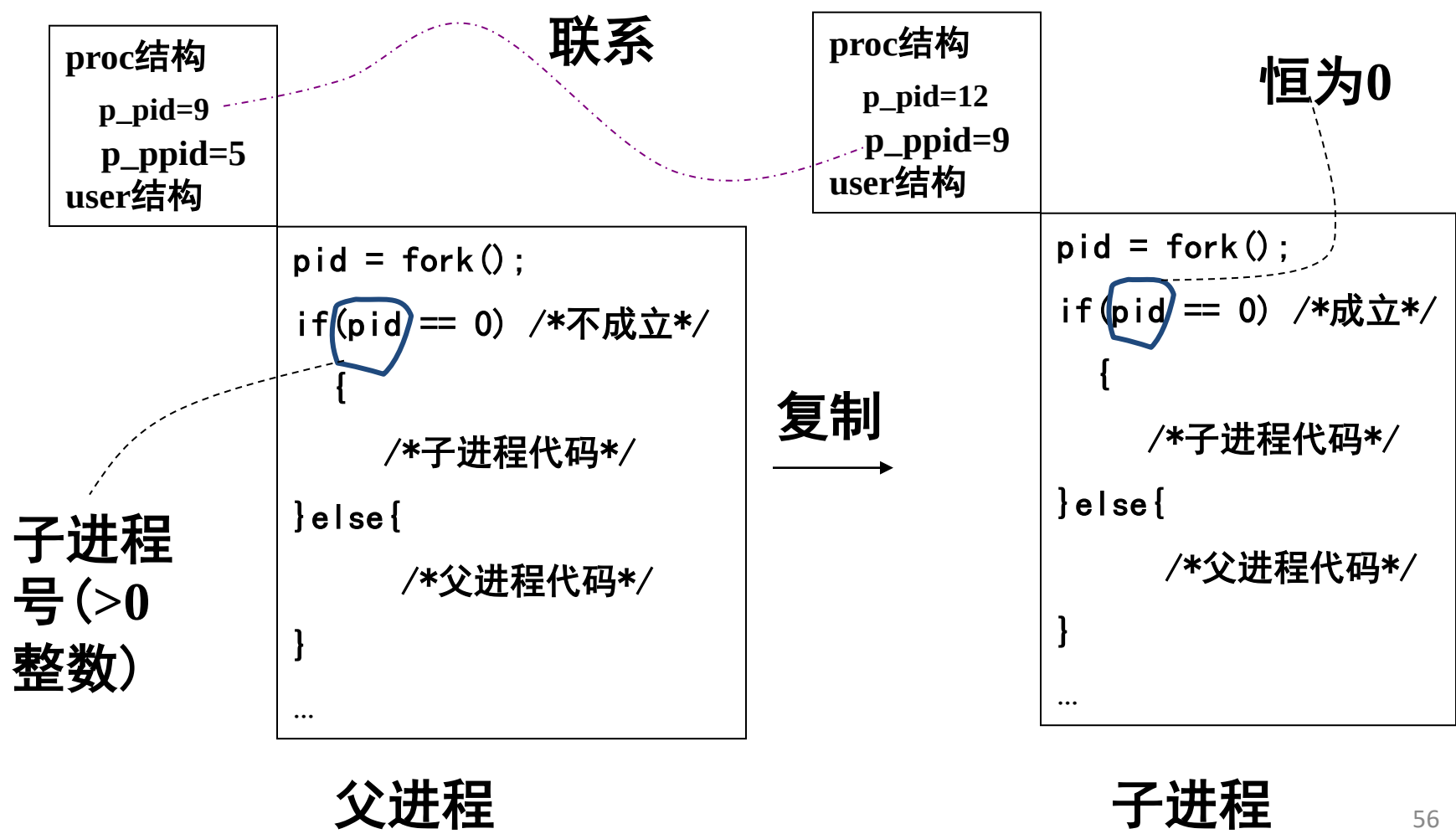
UNIX有关进程的系统调用命令

- 进程自我结束
 - `exit(status)`
 - `status`为终止状态
 - 进程自我终止，通知父进程，唤醒父进程
 - 系统做善后处理：收回存储空间，撤销PCB
- 等待子进程终止
 - `pid=wait(&status)`
 - 返回终止子进程编号
 - 参数为子进程的终止状态

UNIX有关进程的系统调用命令

- **Abort终止进程**
- **只有被终止的进程的父进程才能执行此命令**
- **父进程终止子进程的原因**
 - 子进程使用过量资源
 - 分配给子进程的任务过时
 - 父进程退出时OS不允许其子进程继续，但在UNIX系统中，当父进程终止，其所有子进程会以init进程作为新的父进程
 - **除正常终止外，进程地址越界，非法指令，来自用户或父进程的kill指令，都可以导致非正常终止**

fork() 创建子进程



execl() 加载并执行新程序

proc结构
p_pid=9
p_ppid=5
user结构

```
pid=fork();
if(pid==0) /*不成立*/
{
    execl("P",0);
}else{
    /*父进程代码*/
}
...
```

父进程

proc结构
p_pid=12
p_ppid=9
user结构

```
pid=fork();
if(pid==0) /*成立*/
{
    execl("P",0);
}else{
    /*父进程代码*/
}
...
```

子进程

execl() 加载并执行新程序

proc结构
p_pid=9
p_ppid=5
user结构

```
pid=fork();
if(pid==0) /*不成立*/
{
    execl("P",0);
}else{
    /*父进程代码*/
}
...
```

父进程

proc结构
p_pid=12
p_ppid=9
user结构

程序P（覆盖原来程序）

子进程

exit() 进程自我终止

proc结构
p_pid=9
p_ppid=5
user结构

```
pid=fork();
if(pid==0) /*不成立*/
{
    execl("P",0);
}else{
    /*父进程代码*/
    id=wait(&s)
}
...
```

父进程

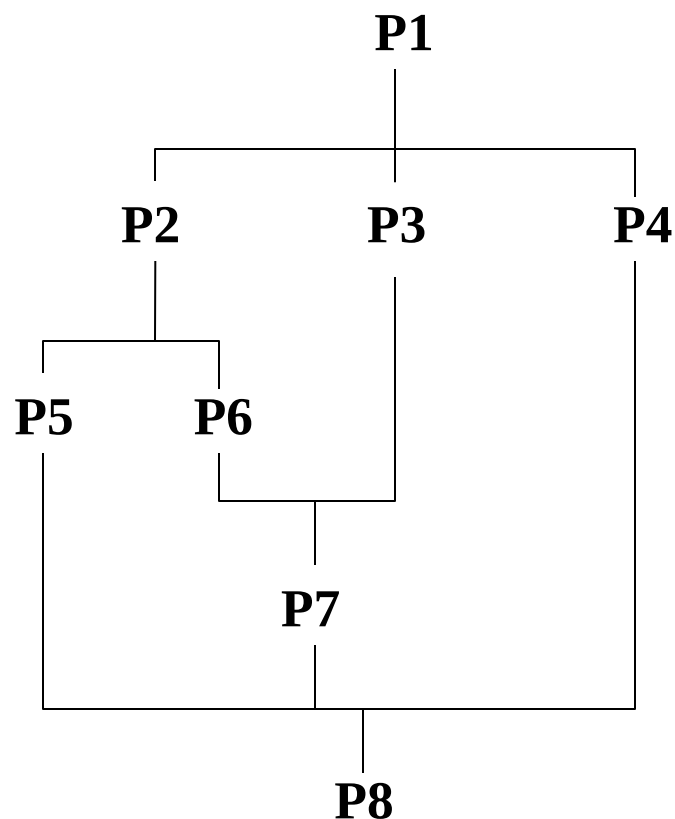
proc结构
p_pid=12
p_ppid=9
user结构

程序P（覆盖原来程序）

exit(2)

子进程

例子: 设有8个程序，执行次序如下图所示，试用 **fork,execl,wait,exit** 系统调用描述之。



```
main()
{
    int pid1,pid2,pid3,pid4,pid5,pid6,pid7,pid8;
    int end_p1=end_p2=end_p3=end_p4=end_p5=...=end_p8=0;
    int pid, status;

    if((pid1=fork())==0) execl("P1",0);
    wait(&status);
    if((pid2=fork())==0) execl("P2",0);
    if((pid3=fork())==0) execl("P3",0);
    if((pid4=fork())==0) execl("P4",0);
    do{        //等待P2结束
        pid=wait(&status);
        if(pid==pid2) end_p2=1;
        if(pid==pid3) end_p3=1;
        if(pid==pid4) end_p4=1;
    }while(end_p2==0);
```

```
if((pid5=fork())==0) execl("P5",0);
if((pid6=fork())==0) execl("P6",0);
do{    //等待P3和P6结束
    pid=wait(&status);
    if(pid==pid3) end_p3=1;
    if(pid==pid4) end_p4=1;
    if(pid==pid5) end_p5=1;
    if(pid==pid6) end_p6=1;
}while(end_p3==0||end_p6==0);
if((pid7=fork())==0) execl("P7",0);
do{    //等待P4,P5,P7结束
    pid=wait(&status);
    if(pid==pid4) end_p4=1;
    if(pid==pid5) end_p5=1;
    if(pid==pid7) end_p7=1;
}while(end_p4==0||end_p5==0||end_p7==0);
if((pid8=fork())==0) execl("P8",0);
wait(&status);
exit(0);
}
```

fork问题

- *fork* 功能
 - 复制地址空间 (code+data+stack)
 - 复制控制结构 (proc, user)
- 特点
 - 父子进程之间有两个各自独立的数据拷贝
- 问题
 - 不加载新程序
 - 无法实现数据共享, 不能描述诸如有界缓冲区问题
 - 若加载新程序
 - 复制没有意义, 浪费时间和空间

vfork与fork

- vfork
 - 只复制控制结构 (proc+user) ;
 - 不复制地址空间 (code+data)
 - 父子进程共享地址空间
- 使用
 - 父进程使用 vfork 创建子进程;
 - 子进程与父进程共享地址空间;
 - 子进程使用 execve 改变其虚拟地址空间.

2.2.9 进程与程序的联系与差别

- 进程与程序的联系
 - 进程包括一个程序
 - 进程存在的目的就是执行这个程序
- 进程与程序的差别
 - 程序静态，进程动态
 - 程序可长期保存，进程有生存期
 - 一个程序可对应多个进程，一个进程只能执行一个程序

■ 思考

- 若系统中没有运行进程，是否一定没有就绪进程？
- 若系统中即没有运行进程，也没有就绪进程，系统中是否就没有进程？
- 在采用优先级进行调度时，运行进程是否一定是系统中优先级最高的进程？
- 某进程被唤醒后立即投入运行，就说这个系统采用的是剥夺式调度方法？

提纲

2.1 多道程序设计

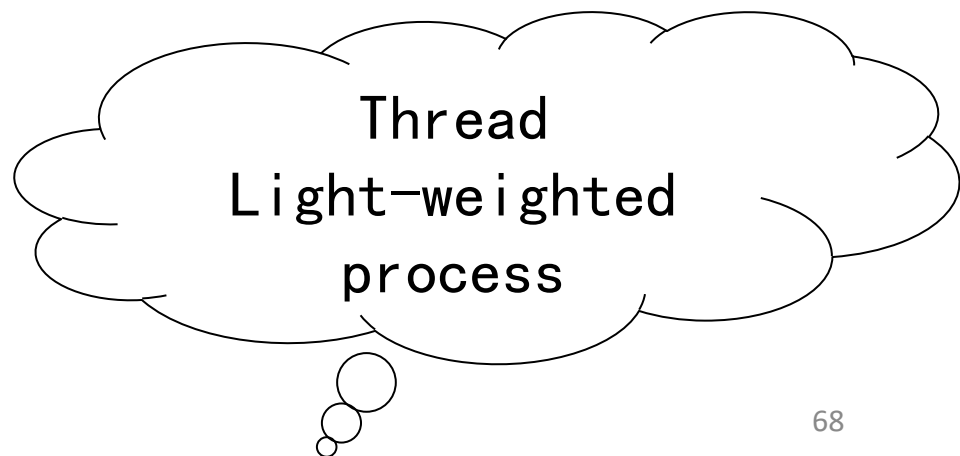
2.2 进程的引入

2.3 线程与轻进程

2.4 作业

2.3 线程与轻进程

- 2.3.1 线程的引入
- 2.3.2 线程的概念
- 2.3.3 线程的结构
- 2.3.4 线程控制块
- 2.3.5 线程的实现
- 2.3.6 线程的应用
- 2.3.7 Java线程



2.3.1 线程的引入

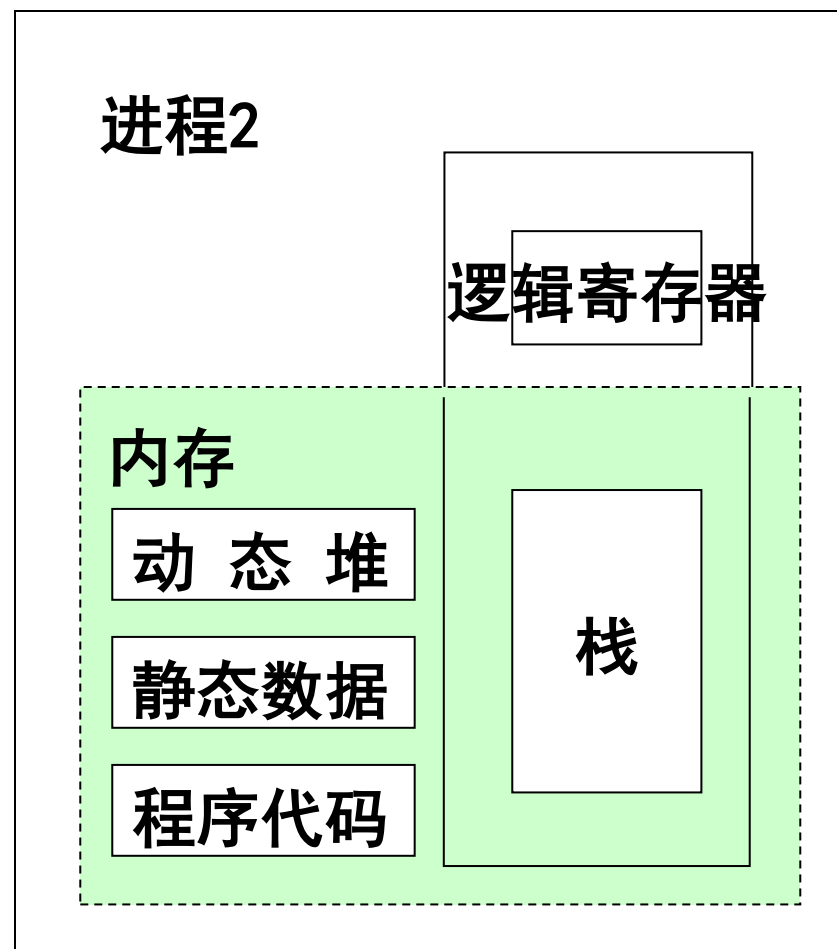
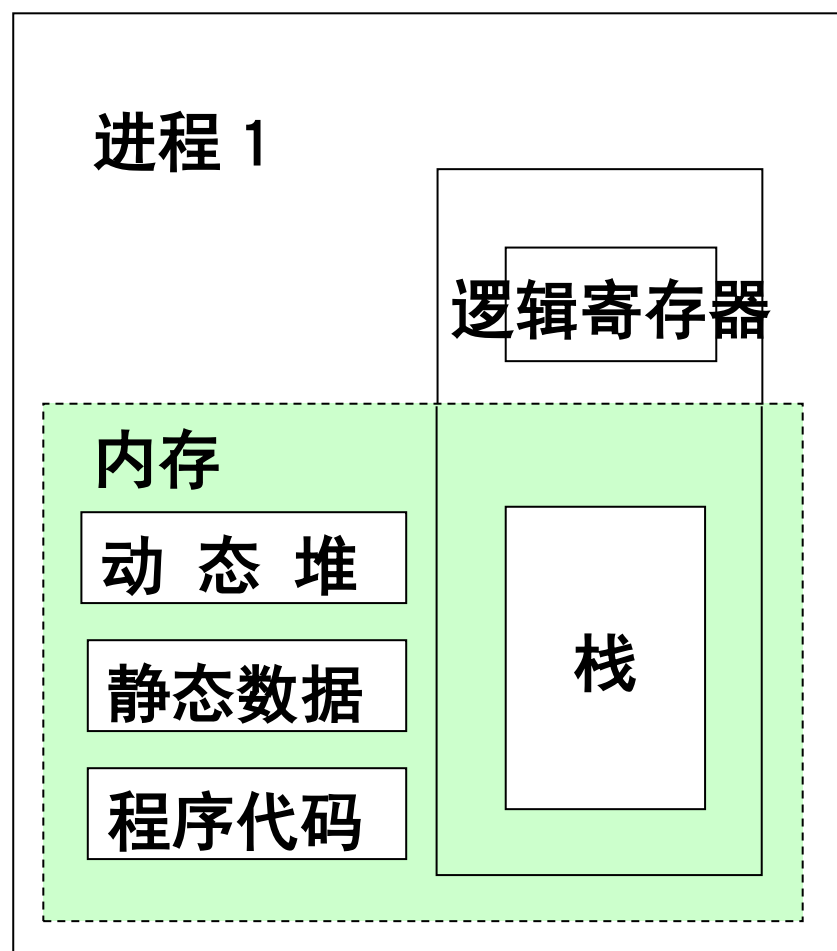
- 进程切换
 - 上下文涉及内容多，开销大，“笨重”
 - PCB+程序
 - 系统环境：地址空间，系统栈，打开文件表，
 - 相关进程之间耦合关系差(无共享的地址空间)
- 解决方案
 - Multi-threading
 - 同一进程中包含多个线程
 - 上下文只涉及寄存器和用户栈，切换速度快
 - 相关线程之间通讯方便、快捷
 - 多个线程共享CPU可以实现并发运行

2.3.2 线程的概念

- 进程中一个相对独立的执行流。
- 进程 vs. 线程
 - 进程是资源分配单位
 - 线程是执行单位
- 多线程优点
 - 切换速度快（地址空间不变）（light weighted）
 - 系统开销小
 - 通讯容易（共享数据空间）

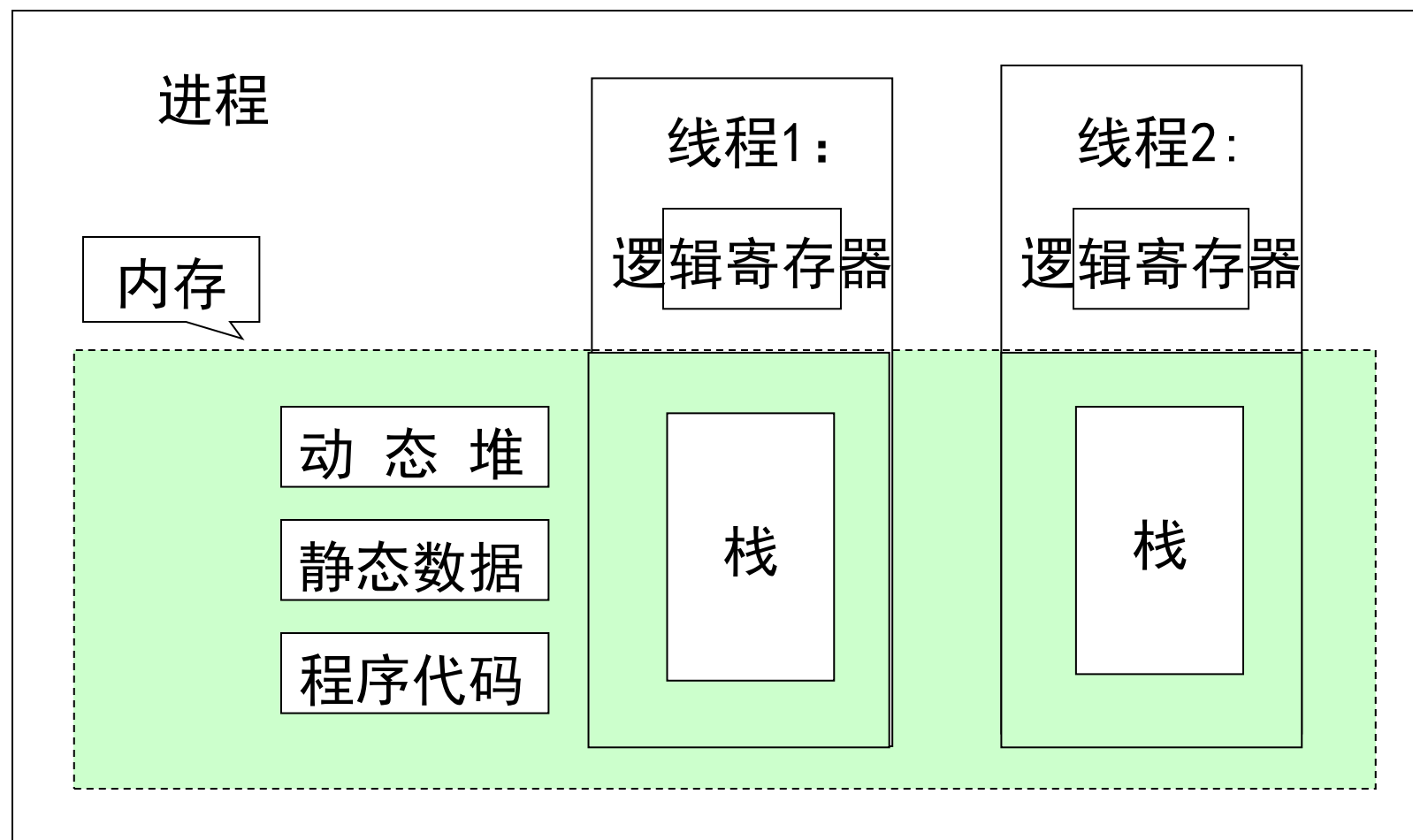
2.3.3 线程结构

多进程结构（用户视图）



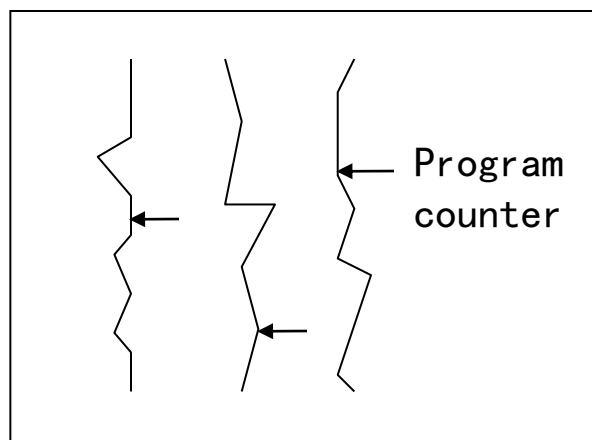
2.3.3 线程结构(Cont.)

多线程结构（用户视图）

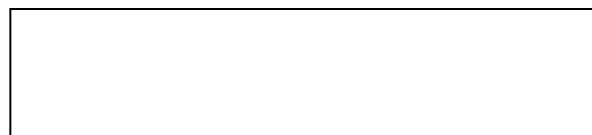


2.3.3 线程结构(另一种表示)

Task:



text segment



data segment

Solaris系统

2.3.4 线程控制块

- TCB(Thread control block)
 - 标志线程存在的数据结构，其中包含对线程管理需要的全部信息
- 内容
 - 线程标志
 - 线程状态
 - 调度参数
 - 现场(通用寄存器, PC, SP)
 - 链接指针
- 存放位置
 - 用户级线程：目态空间
 - 核心级线程：系统空间

线程操作

- 在进程创建时，同时为该进程创建第一个线程，用以运行程序，在以后适当的时候，通过“线程创建”系统调用创建线程
- 线程共享进程的代码段和数据段。但是线程有各自的用户栈，可以独立调用，占用CPU运行
- 当线程被剥夺处理器时，只需将其线程现场保存在该线程对应的栈区
- 当进程内的所有线程结束时，意味着进程结束，从而释放进程所占用的所有资源。

2.3.5 线程的实现

- 2.3.1 用户级别线程
 - User-level thread
- 2.3.2 核心级别线程
 - Kernel-level thread
- 2.3.3 混合线程
 - Hybrid approach

2.3.5.1 用户级别线程

- 实现方法：
 - 基于library函数，系统不可见
 - 线程创建、撤销、状态转换在目态完成
 - TCB在用户空间，每个进程一个系统栈.

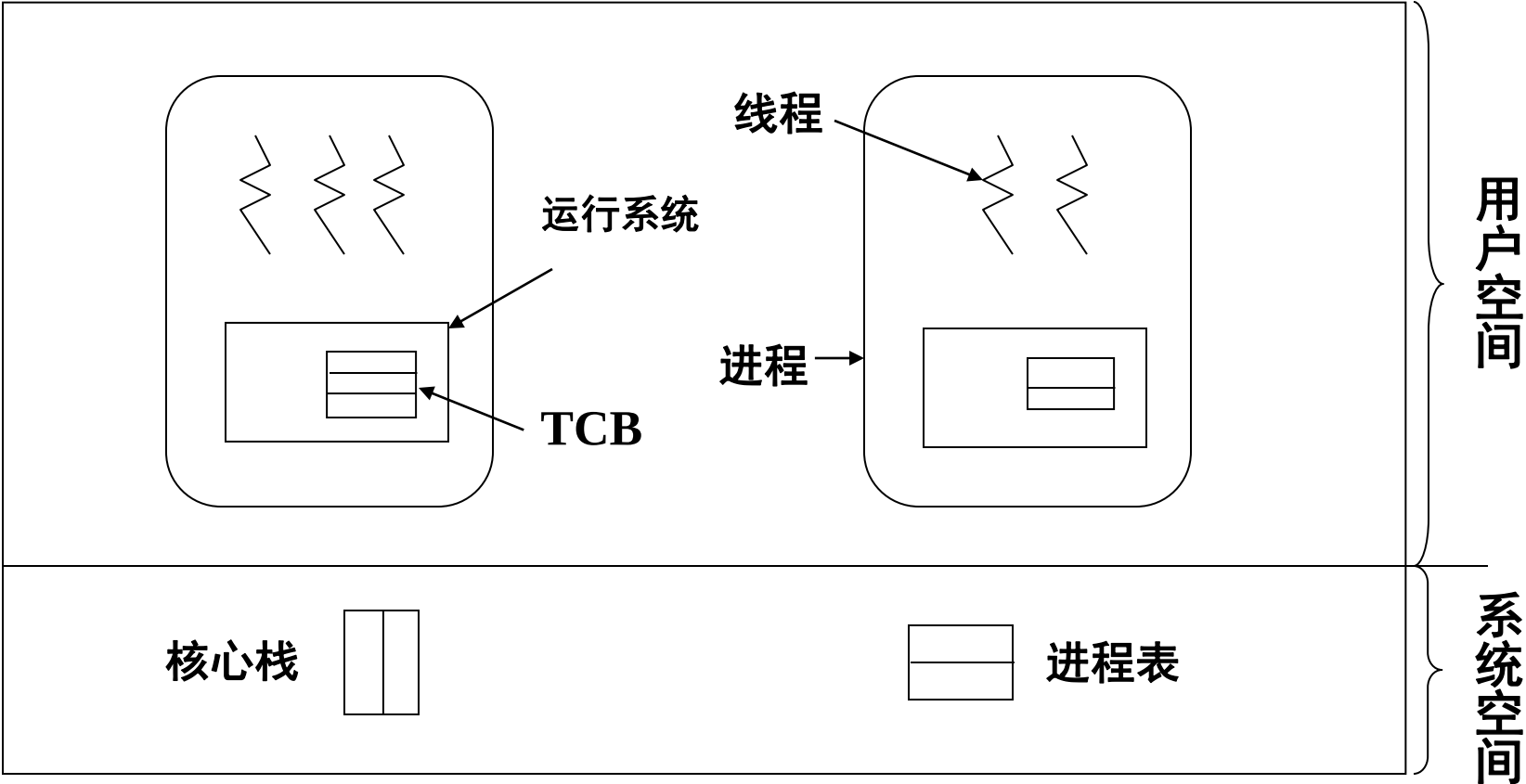
2.3.5.1 用户级别线程(Cont.)

- 优点：
 - 不依赖于操作系统，调度灵活
 - 同一进程中多线程切换速度快(不需进入操作系统)
- 缺点：
 - 同一进程中多个线程不能真正并行，即使在多处理器环境中(线程对操作系统不可见)
 - 一个线程进入系统受阻，进程中其它线程不能执行，不能实现真正并行。

2.3.5.1 用户级别线程(Cont.)

- 线程状态和进程状态
 - 若同一进程中的多个线程至少有一个处于运行态，则该进程的状态为运行态
 - 若同一进程中的多个线程均不处于运行态，但至少有一个线程处于就绪态，则该进程的状态为就绪态
 - 若同一进程中的多个线程均处于等待态，则该进程的状态为等待态

2.3.5.1 用户级别线程(Cont.)



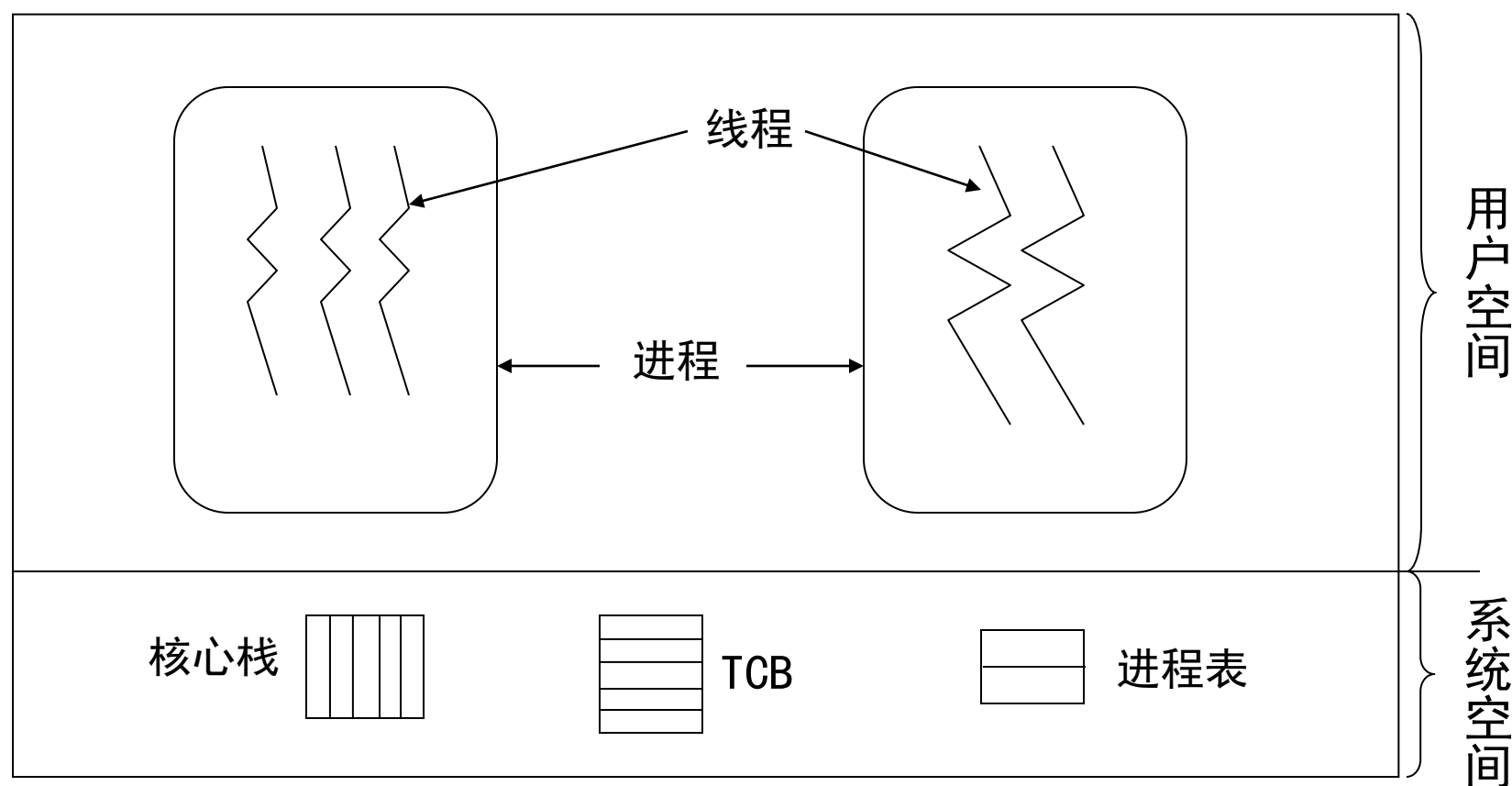
2.3.5.2 核心级别线程

- 实现方法：
 - 基于系统调用由操作系统创建
 - 创建、撤销、状态转换由操作系统完成
 - 线程是处理器调度的基本单位
 - TCB在操作系统空间，每个线程一个系统栈
 - 对于核心级别线程，进程状态不具有实际意义

2.3.5.2 核心级别线程(Cont.)

- 优点：
 - 同一进程内多线程可以并行执行（多CPU）
 - 进程中的一个线程被阻塞了，内核可以调度同一进程的其他线程占有处理器运行
- 缺点：
 - 线程的控制和状态转换需要进入操作系统，系统开销大
 - 控制由系统实现，调度算法不能灵活控制

2.3.5.2 核心级别线程(Cont.)



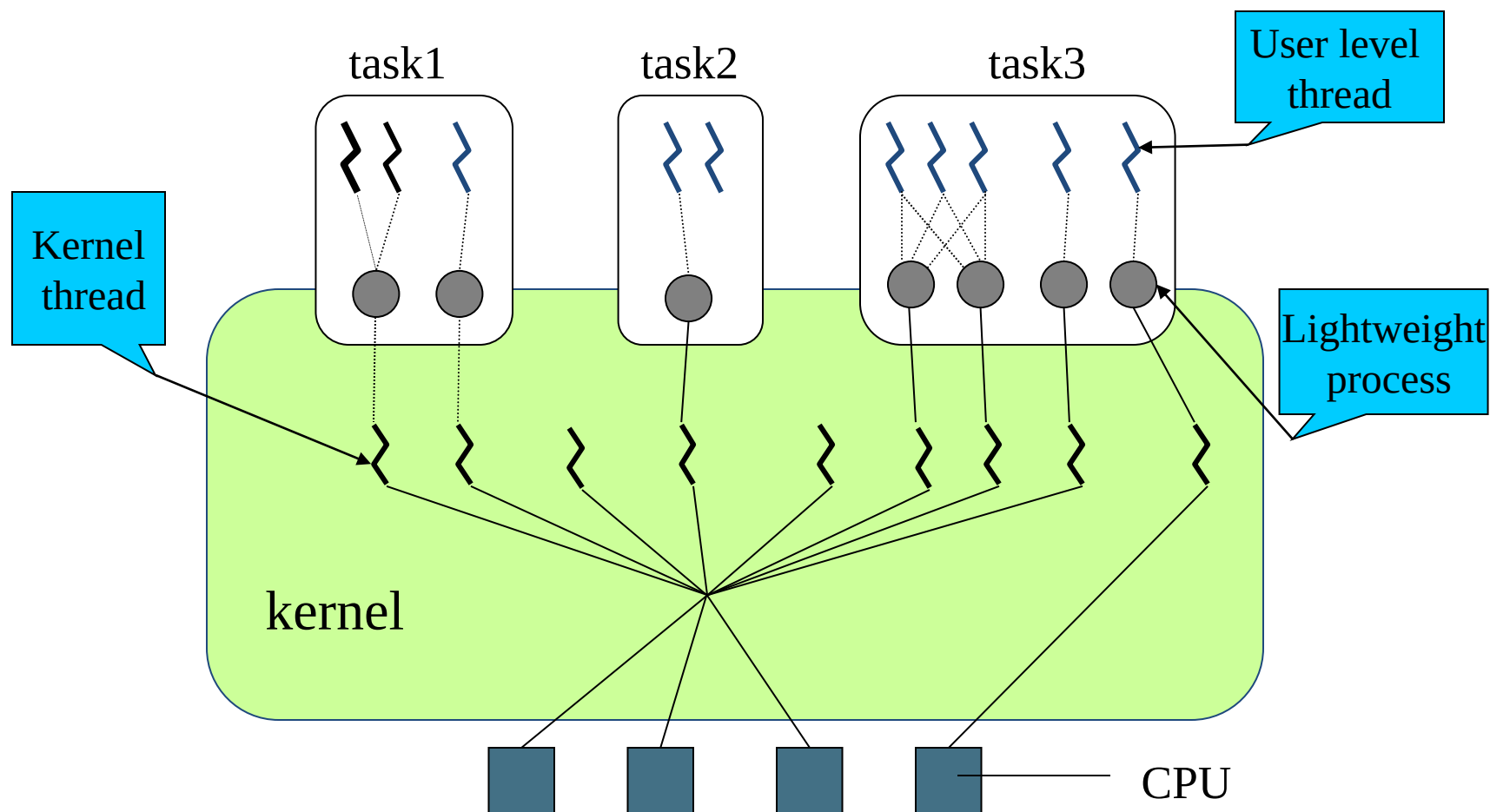
2.3.5.2 核心级别线程(Cont.)

- 用户级别线程和核心级别线程区别
 - 核心级别线程操作系统**可见**；用户级别线程操作系统**不可见**
 - 用户级别线程的创建、撤销和调度**不需要**操作系统的支持，且是在（程序）语言这一级处理的。核心级别线程的创建、撤销和调度都**需要**操作系统内核的支持
 - 用户级别线程执行系统调用命令将导致其所属**进程**被中断，核心级别线程执行系统调用命令将导致**线程**被中断
 - 在仅有用户线程的系统内，CPU调度以**进程**为单位。在有核心级别线程的系统内，CPU调度以**线程**为单位
 - 用户级别线程的实体是运行在**用户态**下的程序，而核心级别线程的实体则是可以运行在**任何状态**下的程序

2.3.5.3 混合线程

- Solaris系统
 - User level thread
 - 由Lib程序支持（创建, 调度）
 - Light weighted process (LWP)
 - 由Lib程序支持
 - 每个task至少一个LWP
 - 用户级别线程与LWP可以多对多
 - LWP对操作系统可见
 - 只有与LWP相联系的用户线程向前推进
 - Kernel level thread
 - 由kernel 支持
 - 每个LWP与唯一一个核心线程对应
 - 核心线程可与CPU多对多，可对一
 - 多个用户级别线程可以通过LWP映射到一个核心级别线程

2.3.5.3 混合线程(Solaris)



2.3.6 线程的应用

- 内在的多控制流，具有合作性质，需要共享数据
 - 生产—消费问题：通过共享存储空间实现通信
- 多线程优于多进程
 - 建立速度快100倍！
 - 切换速度也有数量级之差
- 提高处理器与设备的并行性
 - 一个线程等待，其它线程可继续执行
- 多处理器环境
 - 多线程可以并行执行，提高资源利用率，加快进程推进速度

2.3.6 线程的应用(Cont.)

- 例子：
 - Word字处理（不同代码）
 - 交互编辑 (T1)
 - 词法检查 (T2)
 - 定时保存 (T3)
 - HTTP server（相同代码）
 - 每个http请求，对应一个线程
 - 这些线程执行相同的程序，对应同一进程.

暴风影音（在线看电影）



并发的功能：视频解码，音频解码，网络接收
每个功能都创建为一个线程，从而实现多功能的并发。

2.3.6 线程的应用(Cont.)

- 线程和进程的比较
 - 拥有的资源
 - 调度
 - 并发性
 - 安全性

2.3.6 线程的应用(Cont.)

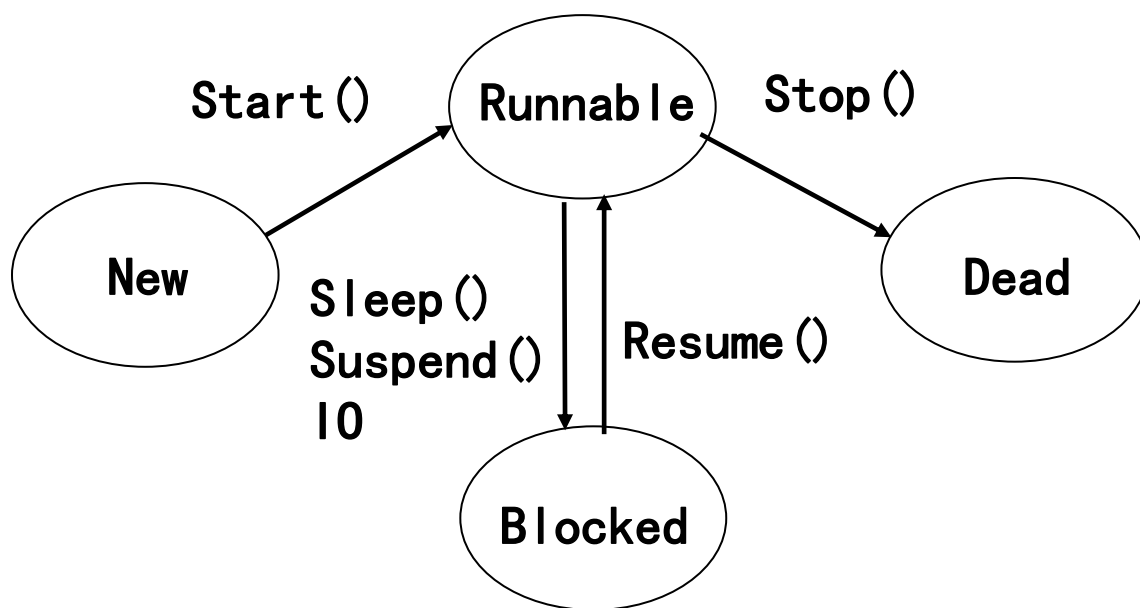
- 多线程与多任务的区别：
 - 多任务是针对操作系统而言的，代表操作系统可以同时执行的程序个数
 - 多线程是针对一个进程而言的，代表一个进程内部可以同时执行的线程个数，每个线程完成不同的任务.

例1：Java线程

- Java线程可以通过扩展线程类(thread)来实现
- Java线程四种基本状态
 - New：新建的线程
 - Runnable：可运行状态
 - Blocked：封锁状态
 - Dead：终止状态。

Java线程

- Java线程状态之间的转换关系图



JAVA线程与JVM

- Java线程是由Java虚拟机JVM支持的
- JVM位于操作系统之上
- Java线程与操作系统线程之间的对应关系由JVM确定
 - 对于Windows NT的JVM, Java线程与操作系统线程具有一对一关系
 - 对于Solaris的JVM, 其对应关系为多对多.

例2: Linux 线程

■ Clone系统调用的形式

- `pid = clone(function, stack_ptr, sharing_flag, arg)`
 - `function`为新线程执行的代码
 - `stack_ptr`为新线程栈指针
 - `arg`为调用参数
 - `Sharing-flag`表示共享标志位:
 - `CLONE_VM`, `CLONE_FILES`, `CLONE_SIGHAND`, `CLONE_PID`
 - `CLONE_VM`是否共享存储区, 为1表示线程和线程之间共享存储区
 - `CLONE_FILES`表示是否共享文件(如进程打开文件, 线程是否可用)
 - `CLONE_SIGHAND`表示线程是否共享进程的消息处理机制
 - `CLONE_PID`表示线程是否继承进程的id号

例3：Windows进程、线程与纤程

■ 进程

■ 在Windows环境中创建进程/线程的过程

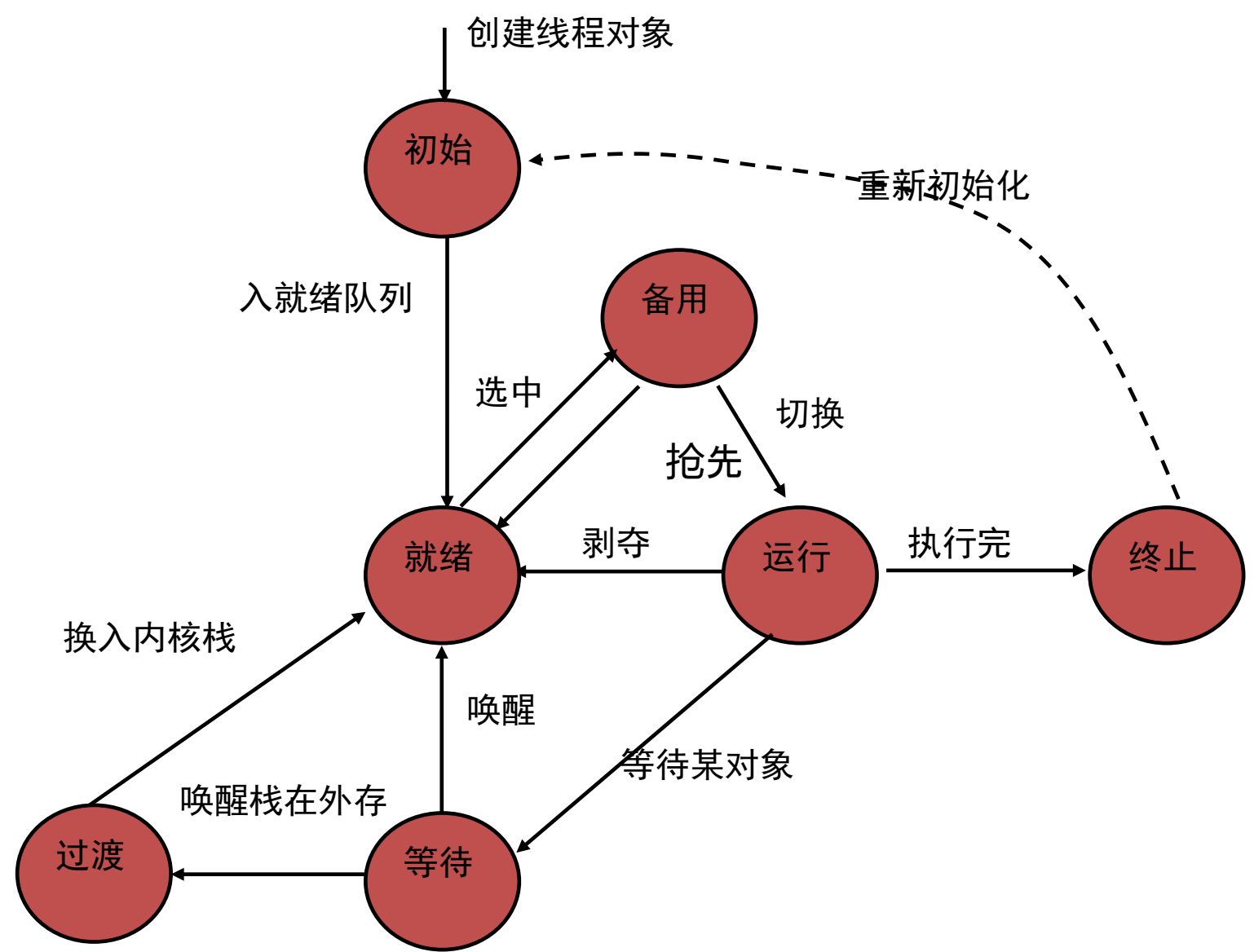
- 当Windows应用执行CreateProcess调用，消息被发给Windows子系统，Windows子系统调用进程管理器创建进程，进程管理器调用OM创建进程对象，然后返回对象把柄给Windows
- Windows子系统再次调用进程管理器为该进程创建线程，最后Windows将把柄返给新进程和线程
- 创建线程createThread调用.

进程对象
描述图

对象头部属性	Type: Process
对象体属性	进程标识 (Process id) 访问令牌 (Access token) 基础优先级 (Base priority) 缺省亲合处理机 (Default processor affinity) 配额限制 (Quota limits) 执行时间 (Execution time) 输入/输出记数 (I/O counters) 执行/调试端口 (Exception/debugging ports) 退出状态 (exit status)
服务	创建进程 (Create process) 打开进程 (Open process) 查询进程信息 (Query process information) 设置进程信息 (Set process information) 当前进程 (Current process) 终止进程 (Terminate process) 分配/释放虚拟存储 (Allocate/free virtual memory) 读/写虚拟存储 (Read/write virtual memory) 保护虚拟存储 (Protect virtual memory) 加锁/开锁虚存 (Lock/unlock virtual memory) 查询虚拟存储 (Query virtual memory) 刷新虚拟存储 (Flush virtual memory)

线程对象 描述图	对象头部属性 属性	类型：线程（Type: thread）
	属性	客户标识（client id） 线程上下文（thread context） 动态优先级（dynamic priority） 处理机亲合掩码（thread processor affinity） 已执行时间（thread execution time） 警觉状态（alert status） 挂起记数（suspension count） 非角色令牌（impersonation token） 终止端口（termination port） 终止状态（exit status）
	服务	创建线程（Create thread） 打开线程（Open thread） 查询线程状态（Query thread information） 设置线程状态（Set thread information） 当前线程（Current thread） 终止线程（Terminate thread） 取上下文（Get context） 置上下文（Set context） 挂起（Suspend） 恢复（Resume） 警示线程（Alert thread） 测试线程警示（Test thread alert） 注册终止端口（Register termination port）

- Windows线程状态
 - 初始
 - 就绪
 - 备用：已经选择好线程的执行处理器，正在等待切换以进入运行状态。每个处理器上只有一个处于备用状态的进程
 - 运行
 - 等待
 - 过渡：与就绪态类似，但是，其核心堆栈位于外存储器
 - 终止



Windows线程状态转换图

提纲

2.1 多道程序设计

2.2 进程的引入

2.3 线程与轻进程

2.4 作业

2.4 作业 (Job)

- 作业概念
 - 用户要求计算机系统为其完成的计算任务集合。
- 作业步 (job step)
 - 作业处理过程中一个相对独立的步骤
 - 一般一个作业步可由一个进程完成
 - 某些作业步之间可以并行
- 作业分类
 - 批处理作业
 - 交互式作业.

2.4 作业 (Cont.)

- 作业控制块
 - 标志作业存在的数据结构，其中包含对作业进行管理所需要的全部信息
 - 如作业名称、作业状态、调度参数、资源请求、相关进程、作业长度、在输入井与输出井中的存放位置、记账信息等

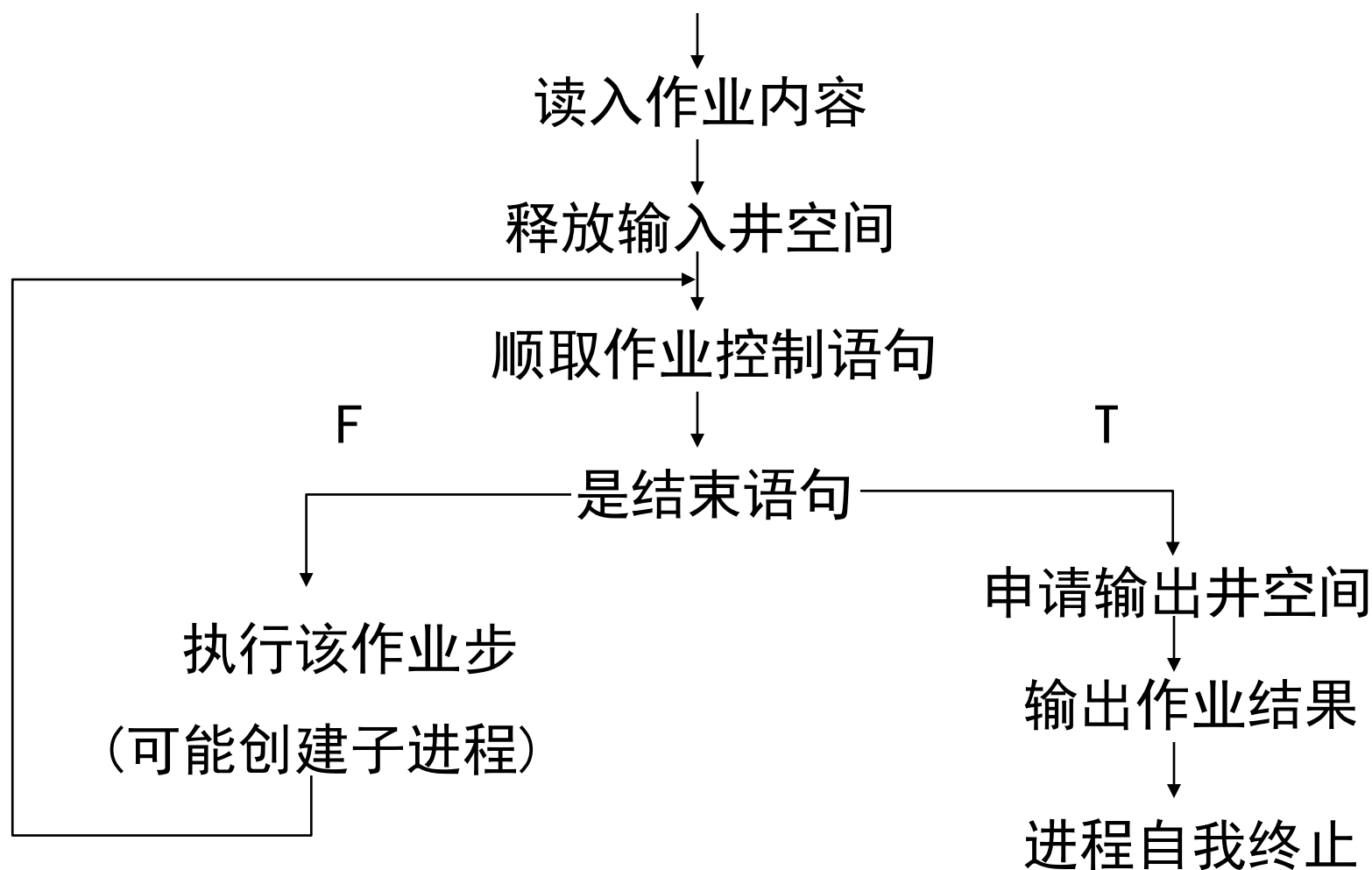
2.4.1 批处理作业

- 作业控制语言 (JCL)
 - 描述批处理作业控制意图的语言
- 作业说明书 (JCL 语句的序列)
 - 一般以特殊符号起始
 - \$JOB J1
 - \$FORTN ...
 - \$LINK ...
 - \$EXEC ...
 - \$ENDJOB
- 作业控制程序
 - 解释并处理作业说明书的程序
- 作业控制进程
 - 执行作业控制程序的进程.

2.4.1 批处理作业

- 批处理作业的工作过程
 - 作业由假脱机输入程序控制进入输入井
 - 经由操作系统的作业调度程序选中进入内存
 - 同时由作业调度程序为其建立作业控制进程执行作业控制程序
 - 作业控制进程解读作业说明书中的语句, 并根据作业步的要求为其建立相应的进程

作业控制进程



2.4.2 交互式作业

- 帐户管理

- /etc/passwd文件(创建用户时, 系统将设置口令文件)
- passwd中包括注册用户的所有信息(用户名, 口令, 用户根目录, 同组用户, 余额...), 为系统文件

- 创建与撤销

- 创建:

- 用户提供(用户名, 口令)
- 系统操作员建立(根目录/usr/zhang, 填写passwd文件)

- 撤销:

- 删除该用户目录及所有文件
- 在passwd文件中清除对应entry

2.4.2 交互式作业(Cont.)

- 注册与注销

- 注册:

- logon: 用户名
 - password: *****

- (使用)

- 注销:

- 显式注销:

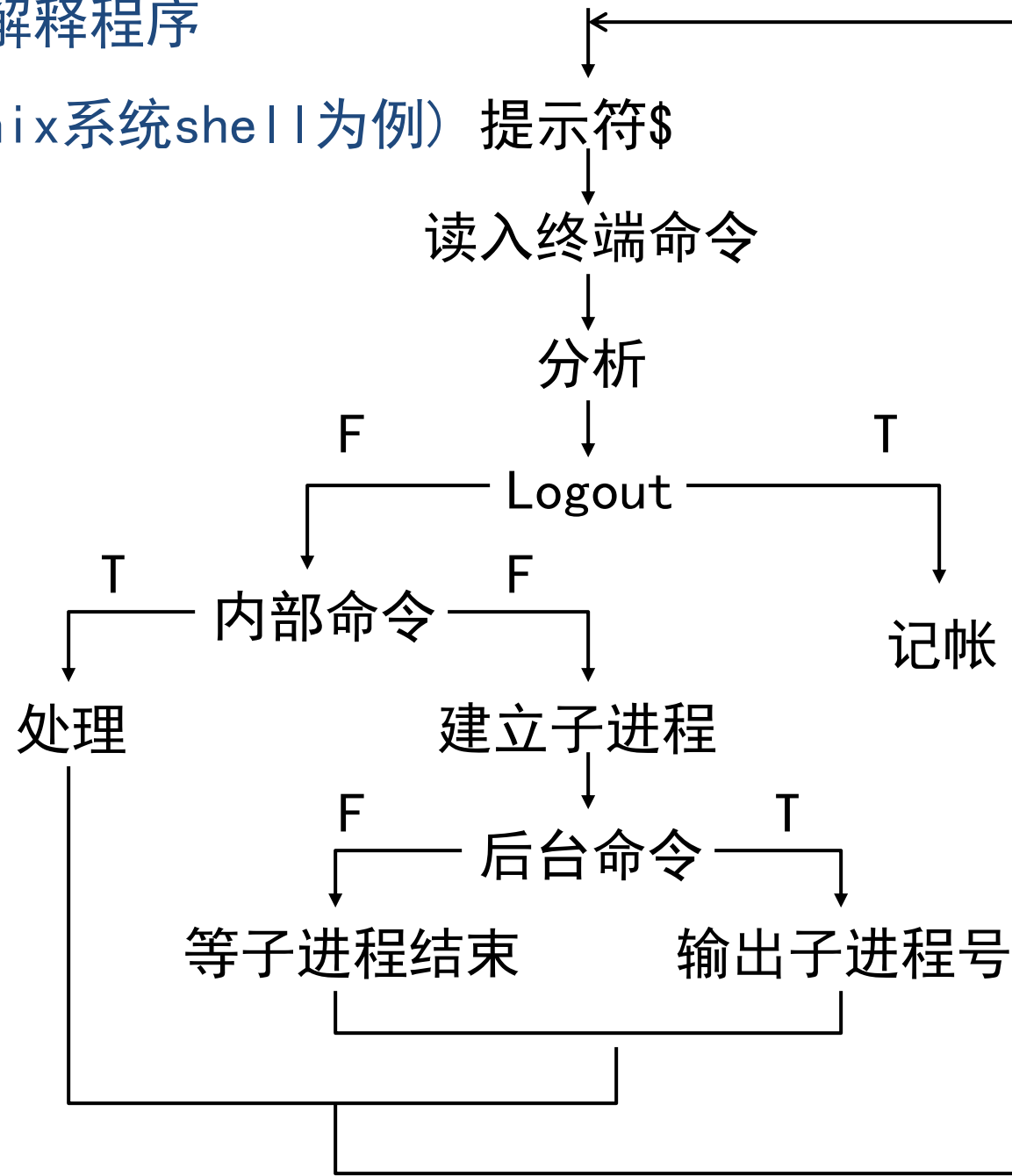
- Logoff (用户主动退出)

- 隐式注销:

- (如5分钟无输入命令)

命令解释程序

(以Unix系统shell为例) 提示符\$



小结：作业、进程、线程

- 作业与进程
 - 作业进入内存后变为进程
 - 一个作业通常与多个进程相对应
- 进程与线程
 - 一个进程一般包含多个线程，至少包含一个线程
 - 不支持多线程的系统，可视为单线程进程